Površina unije pravougaonika

Lana Matić 1009/2025
Matematički fakultet Univerziteta u Beogradu

1 Uvod

Izračunavanje površine unije geometrijskih objekata jedan je od osnovnih problema u računarskoj geometriji. Problem se javlja u brojnim praktičnim primenama: u geografskim informacionim sistemima, u računarskoj grafici, u VLSI dizajnu integrisanih kola, kao i u robotici pri planiranju kretanja i analizi prostora.

U opštem slučaju, određivanje površine unije proizvoljnih poligona je složen problem. Međutim, kada su objekti pravougaonici čije su stranice paralelne koordinatnim osama (eng. *axis-aligned rectangles*), problem dopušta efikasna rešenja zasnovana na tehnici brišuće prave (eng. *sweep line*).

U ovom radu prikazan je algoritam brišuće prave sa kompresijom koordinata za rešavanje problema *Rectangle Area II* (LeetCode 850).

2 Opis problema

Dat je niz od N pravougaonika u ravni čije su stranice paralelne koordinatnim osama. Svaki pravougaonik je definisan sa četiri celobrojna parametra:

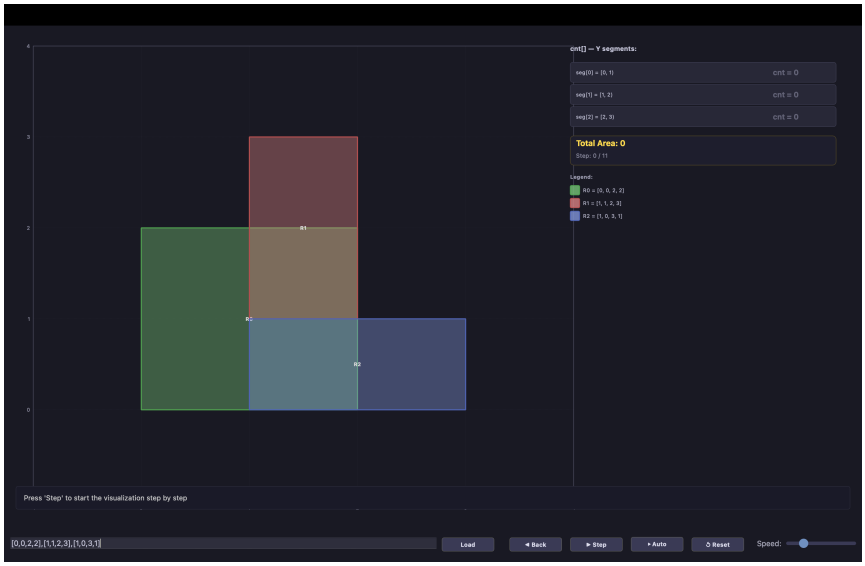
$$\texttt{rectangles}[i] = [x_1,\ y_1,\ x_2,\ y_2]$$

gde je (x_1, y_1) donji levi ugao, a (x_2, y_2) gornji desni ugao.

Zadatak: Izračunati **ukupnu površinu** koju pokrivaju svi pravougaonici, pri čemu se svaka tačka ravni broji **samo jednom**, čak i ako je pokrivena sa više pravougaonika. Rezultat vratiti po modulu $10^9 + 7$.

2.1 Primer

Na slici 1 su prikazana tri pravougaonika koja se delimično preklapaju. Naivnim sabiranjem površina dobijamo $4 + 2 + 2 = 8$, ali je tačan odgovor 6 jer se preklapajuće oblasti broje samo jednom.



Slika 1: Ulaz: `[[0,0,2,2], [1,1,2,3], [1,0,3,1]]`. Površina po modulu $10^9 + 7$ je **6**.

3 Algoritam: Brišuća prava sa kompresijom koordinata

Za rešenje problema korišćen je algoritam brišuće prave koja se kreće s leva na desno duž x-ose uz kompresiju koordinata. U svakom trenutku, brišuća prava preseca određene pravougaonike. Delovi linije koji su unutar bar jednog pravougaonika čine *aktivnu visinu*.

Površina se računa kao suma proizvoda širine svake vertikalne trake i njene aktivne visine:

$$\text{Ukupna površina} = \sum_{\text{trake}} \underbrace{(x_{i+1} - x_i)}_{\text{širina}} \cdot \underbrace{h_{\text{aktivna}}}_{\text{visina}}$$

3.1 Korak 1: Kompresija Y koordinata

Iz svih pravougaonika izvlačimo jedinstvene Y koordinate i sortiramo ih. One definišu Y-*segmente* - intervale čija se pokrivenost prati tokom prelaza.

Za primer sa slike 1:

$$Y = \{0, 1, 2, 3\} \quad \Rightarrow \quad \text{segmenti: } [0, 1),\ [1, 2),\ [2, 3)$$

Niz `cnt`$[i]$ broji koliko pravougaonika trenutno pokriva segment i. Segment je *aktivan* ako je `cnt`$[i] > 0$.

3.2 Korak 2: Kreiranje događaja

Za svaki pravougaonik $[x_1, y_1, x_2, y_2]$ kreiramo dva *događaja* (events):

- **START** na $x = x_1$ - pravougaonik počinje da pokriva interval $[y_1, y_2)$
- **END** na $x = x_2$ - pravougaonik prestaje da pokriva interval $[y_1, y_2)$

Događaji se sortiraju po x-koordinati.

3.3 Korak 3: Obrada događaja

Prolazimo kroz događaje s leva na desno. Za svaku traku između dva uzastopna događaja:

1. Izračunamo *aktivnu visinu* - sumu realnih visina svih segmenata sa `cnt`$[i] > 0$
2. Pomnožimo širinu trake sa aktivnom visinom i dodamo u ukupnu površinu
3. Ažuriramo niz `cnt`[] na osnovu tekućeg događaja

4 Analiza složenosti

Neka je N broj pravougaonika (ograničenje: $N \leq 200$).

4.1 Vremenska složenost: $O(N^2)$

Faza	Operacija	Složenost
Kompresija Y koordinata	Sortiranje $2N$ vrednosti	$O(N \log N)$
Kreiranje događaja	$2N$ događaja	$O(N)$
Sortiranje događaja	Sortiranje po x	$O(N \log N)$
Brišuća prava	$2N$ **događaja** $\times$ $O(N)$ **po događaju**	$\boldsymbol{O(N^2)}$

Tabela 1: Vremenska složenost po fazama algoritma.

Kao što je prikazano u tabeli 1, dominantan član je faza brišuće prave. Za svaki od $2N$ događaja:

- Računanje aktivne visine zahteva prolazak kroz ceo `cnt`[] niz dužine $O(N)$
- Ažuriranje `cnt`[] za jedan pravougaonik zahteva prolazak kroz do $O(N)$ segmenata

Dakle: $O(N)$ događaja $\times$ $O(N)$ po događaju $= O(N^2)$.
Za $N = 200$, to je $\sim 80\,000$ operacija - gotovo trenutno.

4.2 Prostorna složenost: $O(N)$

- `yVals`[]: $O(N)$ jedinstvenih koordinata
- `cnt`[]: $O(N)$ segmenata
- `events`[]: $O(N)$ događaja
- `states`[]: $O(N)$ stanja (za vizualizaciju)

4.3 Modularna aritmetika

Koordinate mogu biti do 10^9, pa je maksimalna površina jednog pravougaonika do 10^{18}, što staje u `long long` ($\sim 9.2 \times 10^{18}$). Međutim, zadatak traži rezultat modulo $10^9 + 7$:

$$\texttt{ans} = (\texttt{ans} + (\texttt{width} \bmod M) \cdot (\texttt{activeH} \bmod M)) \bmod M, \quad M = 10^9 + 7$$

Primer primene modularne aritmetike prikazan je na slici 3, gde pravougaonik sa koordinatama do 10^9 daje površinu 10^{18}, a rezultat po modulu je 49.

4.4 Struktura projekta

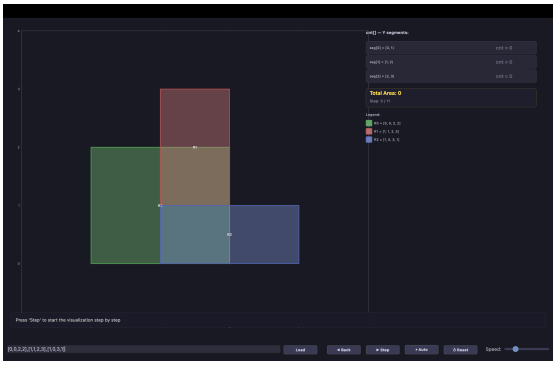
Algoritam je odvojen od vizualizacije - `algorithm_solution.h/cpp` nema nikakvu zavisnost od Qt biblioteke i može se koristiti samostalno ili kao biblioteka za bilo koji frontend.

Fajl	Opis
`algorithm_solution.h/cpp`	Čist C++ — algoritam brišuće prave
`mainwindow.h/cpp/ui`	Qt vizualizacija — crtanje i kontrole
`main.cpp`	Entry point sa tamnom temom
`CMakeLists.txt`	Build konfiguracija

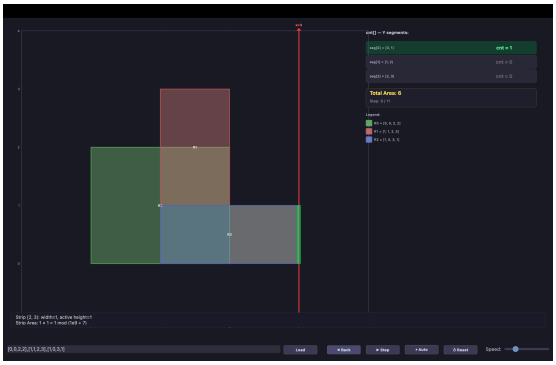
Tabela 2: Struktura projekta.

5 Qt vizualizacija

Vizualizacija omogućava korak-po-korak praćenje rada algoritma. Korisnik unosi pravougaonike u tekstualno polje i koristi kontrole za navigaciju kroz stanja algoritma. Na slici 2 prikazana su dva karakteristična stanja vizualizacije: početno stanje pre pokretanja algoritma (slika 2a) i stanje u koraku 9 od 11, kada je brišuća prava stigla do $x = 3$ (slika 2b).



(a) Početno stanje - pravougaonici učitani, `cnt[]` svi nule, vizualizacija spremna.



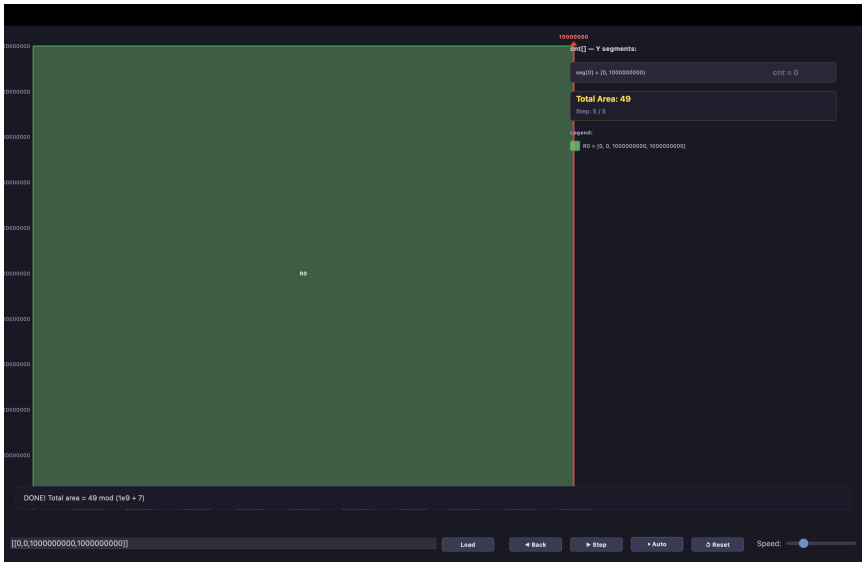
(b) Brišuća prava na x=3 (korak 9/11). Zeleni segment pokazuje aktivan $[0, 1)$. Ukupna površina: 6.

Slika 2: Vizualizacija za primer `[0,0,2,2],[1,1,2,3],[1,0,3,1]`

5.1 Elementi vizualizacije

- **Levi panel** — koordinatna mreža sa obojenim pravougaonicima (providne boje za prikaz preklapanja)
- **Crvena sweep linija** — trenutna x-pozicija, kreće se s leva na desno
- **Žute trake** — oblast čija se površina upravo računa
- **Zeleni segmenti na sweep liniji** — aktivni Y-segmenti ($\texttt{cnt}[i] > 0$)
- **Desni panel** — `cnt[]` niz, zeleno osvetljen kad je segment aktivan
- **Ukupna površina** — tekući zbir sa rezultatom modulo $10^9 + 7$

5.2 Rad sa velikim koordinatama



Slika 3: Ulaz: `[0,0,1000000000,1000000000]`. Površina iznosi 10^{18}, a rezultat po modulu $10^9 + 7$ je **49**. Adaptivna mreža automatski bira odgovarajuće korake za ose.

6 Zaključak

U ovom radu prikazan je algoritam brišuće prave za izračunavanje površine unije pravougaonika čije su stranice paralelne koordinatnim osama. Algoritam koristi kompresiju koordinata kako bi sveo beskonačan koordinatni prostor na konačan skup segmenata, a zatim brišuću pravu za efikasno praćenje pokrivenih oblasti.

Implementacija u jeziku C++ je razdvojena na algoritamski deo (bez zavisnosti od eksternih biblioteka) i vizualizacioni deo (Qt framework), što omogućava nezavisno testiranje i lako proširivanje.

Za ograničenje $N \leq 200$, složenost $O(N^2)$ je potpuno zadovoljavajuća.